

Web Designing - I

• What is HTML?

→ HTML stands for Hypertext Markup Language.

> Markup Language

- It is not a programming language, but a language used to annotate text to define the structure and presentation of content.

> Hypertext

- This refers to the system of hyperlinks (links) that connect web pages to one another, making it possible to navigate the world wide Web.

GSS Notes

> Purpose

- It is the standard language for creating web pages. It defines the meaning and structure of web content - such as headings, paragraphs, lists, images, and links - so a web browser can display it. Think of HTML as the skeleton of a website.

The Building Blocks of HTML

→ HTML documents are made up of key components:

① Elements

- An HTML element is the complete structural unit, generally consisting of an opening tag, the content, and a closing Tag.
e.g: `<P> This is a paragraph </P>`

* What is Web technology

→ Web technology refers to the tools, languages, protocols, and standards used to create, design, host, and manage websites and web applications on the Internet.

It enables:

GSS_Notes

- Online Communication
- Sharing Information
- E-commerce
- Cloud-based applications.

• Evolution of the Web

- Web 1.0 - static web pages (read-only)
- Web 2.0 - Dynamic and interactive (social media, blogs)
- Web 3.0 - Semantic and intelligent web (AI-based)

* World Wide Web (WWW)

→ The World Wide Web (WWW) is a system of interlinked hypertext documents accessed using the Internet.

• Key components of WWW

- ① Web Pages - Documents written in HTML
- ② Web Browsers - Software to view web pages.
- ③ Web servers - Store and deliver web pages.
- ④ URLs - Address of web resources
- ⑤ HTTP/HTTPS - Communication protocols

• Features of WWW

- Hypertext-based
- Multimedia support
- Platform independent
- Distributed system

* Internet

↳ The Internet is a global network of interconnected computers that communicate using standard protocols.

Internet

WWW

Network

Infrastructure

Hardware +
Software

Supports many
services

Service on the
internet

Collection of web
pages

Only web service

* Internet Services

- ① World Wide Web (WWW)
- ② Email
- ③ File Transfer Protocol (FTP)
- ④ Remote Login (Telnet / SSH)
- ⑤ Cloud Computing
- ⑥ Video conferencing
- ⑦ Online streaming

* Web Browsers

↳ A web browser is a software application used to retrieve, present, and navigate web pages.

→ Examples

- Google Chrome
- Mozilla Firefox
- Microsoft Edge
- Safari

→ Functions of a Browser

- Fetch web pages from server
- Interpret HTML
- Apply CSS styles
- Execute Javascript
- Display multimedia content

* Web Server

↳ A web server is hardware or software that stores website files and delivers them

to clients upon request.

> Working of Web Server

- 1) Client sends HTTP request
- 2) Server processes request
- 3) Server sends response (HTML, CSS, images)

> Types of Web Servers

- Apache
- ~~At~~ Nginx
- Microsoft IIS

* Client - Server Architecture

① Core Components GSS_Notes

↳ The model functions through three primary components working in a constant loop:

1) Client

- The front-end device or application (like Google Chrome or a mobile app) that requests resources. It handles the User Interface (UI) and user interactions.

2) Server

- A powerful back-end machine that "listens" for requests. It stores data, executes complex logic, and serves the reque-

sted files (HTML, JSON, Images).

3) Communication Network

- The medium (Internet / LAN) and protocols (HTTP / HTTPS) that allow the two to exchange data.

② The Request - Response Cycle

- When a user interacts with a web application, a sequence of steps occurs across the network:

1) DNS Lookup

- The client asks a DNS server to translate a domain name (e.g; example.com) into an IP address.

2) Handshake

- The client establishes a secure connection (TCP/IP) with the server.

3) The Request

- The client sends an HTTP Request containing:
 - a) Method: (GET, POST, PUT, DELETE)
 - b) Headers: (Browser type, accepted data formats).

4) The Processing

- The server receives the request, queries its database if necessary, and pr-

erases the data.

5) The Response

- The server sends an HTTP Response containing:

a) Status Code: (e.g., 200 OK or 404 not found)

b) Body: The actual content (HTML, CSS, or JS).

* URL (Uniform Resource Locator)

- A URL is the address of a resource on the internet.

> The Anatomy of a URL

- A URL is composed of several distinct parts, each serving a specific purpose in the communication between client and server.

scheme://subdomain.domain.tld:Port/Path?
query#fragment

> Breakdown of Parts:

1) Scheme (Protocol): Defines how data is transferred.

a) http:// (Hypertext Transfer Protocol)

b) https:// (secure version using SSL/TLS encryption)

c) ftp:// (File Transfer Protocol)

2) Subdomain

- A subdivision of the main domain (e.g; blog. or WWW.).

3) Domain Name

- The human-readable name of the server (e.g; google).

4) TLD (Top-Level Domain)

- The Suffix (e.g; .com, .org, .edu).

5) Port

GSS_Notes

- The specific "gate" on the server. Default for HTTP is 80; for HTTPS, it is 443. Usually hidden unless a non-standard port is used.

6) Path

- The specific file or directory on the server (e.g; /products /shoes.html).

7) Query String

- Parameters used to pass data to the server (e.g; ?id=123 & color=blue). it begins with ?.

8) Fragment (Anchor)

- Points to a specific location within the page.

(e.g., # contact). It begins with #.

* Domain Name System (DNS)

↳ DNS converts human-readable domain names into IP addresses.

Computers communicate using numbers (IP addresses). However, humans find it difficult to remember strings of numbers for every website they visit. DNS allows us to use easy to remember names while the network handles the technical routing.

• Levels of DNS Hierarchy

① Root Level (.)

- a) The topmost level in DNS.
- b) Represented by a dot (.) at the end of a domain name
- c) Managed by 13 sets of root servers globally.
- d) It is responsible for directing DNS queries to the correct Top-Level Domain (TLD) servers.

② Top-Level Domain (TLD)

- a) The last part of a domain name.
- b) Examples:
 - i) Generic TLDs (gTLDs): .com, .org, .net, .edu
 - ii) Country Code TLDs (ccTLDs): .np (Nepal),
.uk (United Kingdom)

③ Second-Level Domain (SLD)

- a) The unique name registered under a TLD.
- b) Example: In google.com, google is the SLD.
- c) Represents organizations, businesses, or individuals.

④ Subdomain

- a) Optional prefixes added to organize content or services.
- b) Example:
 - i) mail.google.com → mail is a subdomain.
 - ii) blog.example.com → blog is a subdomain.
- c) Used for different services (mail, blog, shop).

* IP Address

- ↳ In Web Technology, an IP (Internet Protocol) Address is a unique numerical label assigned to every device connected to a computer network that uses the Internet Protocol for communication.

If the URL is the "human-friendly address" (like a name), the IP address is the "actual GPS coordinate" of the server.

Types

- 1) IPv4 - 32-bit (e.g. 192.168.1.1)
- 2) IPv6 - 128-bit (e.g. 2001:db8::1)

* HTTP and HTTPS

↳ In web Technology, HTTP (Hypertext Transfer Protocol) and HTTPS (Secure) are the protocols used to transmit data over the web. While they serve the same ~~functional~~ fundamental purpose - facilitating the request-response cycle - their security mechanisms and performance characteristics differ significantly.

- HTTP (Hypertext Transfer Protocol)

- Stateless protocol
- Used to transfer web data
- Uses port 80

- HTTPS: Adding the "s" for Security

↳ HTTPS is not a ~~sp~~ separate protocol; it is HTTP over TLS (Transport Layer security). It addresses three core security needs:

- 1) Encryption: Scrambles data so eavesdroppers cannot read it (prevents "sniffing").
- 2) Data Integrity: Ensures data isn't tampered with during transit.
- 3) Authentication: Proves that the website is

who claims to be via SSL/TLS certificates.

* Web Page

↳ A web page is a document displayed in a browser, written in HTML.

> Types

1) Static Web Page

- a) Fixed content
- b) HTML only

2) Dynamic Web Page

- a) content changes
- b) Uses scripting and databases

GSS_Notes

① Static web Page Examples

↳ A static web page is like a printed book. The content is hard-coded and stays the same for every single person who views it until the developer manually changes the file.

• Personal Portfolio

- A digital resume or "About Me" page. Since your history and skills don't change every second, these are usually built as static HTML/CSS files.

- Documentation Sites
 - Technical manuals (like the official documentation for a programming language) are often static because the information is the same for all developers.

- Small Business "Brochure" Sites
 - A Local plumber's website that ~~sim~~ simply lists their phone number, services, and address.

② Dynamic Web Page Examples

↳ A dynamic web page changes based on who is looking at it, what time it is, or what the user searches for. These pages are generated "on the fly" by a server.

- E-commerce Sites (Amazon)
 - When you search for "headphones," the page you see is generated specifically for that search. Furthermore, the "Recommended for You" section changes based on your personal shopping history.
- Social Media Feeds (Instagram/x):
 - Your feed is entirely different from your friend's feed. The server pulls your specific "following" list from a database and builds the page for you.

* Website

↳ A website is a collection of related web pages under a single domain.

> Types

- Personal website
- Educational website
- Business website
- E-commerce website
- Government website

* Web Application

↳ A web application is a dynamic application accessed through a browser.

> Examples

- Online banking
- Email systems
- Social Media Platforms.

Qn Write 7 advantages and disadvantages of HTML.

→ 7 advantages

- ① HTML is easy to learn and understand, even for beginners.
- ② It is free to use & does not require any specific software.
- ③ It works on all browsers & operating system.
- ④ It helps create the basic structure of a web page.
- ⑤ HTML files load fast because they are lightweight.
- ⑥ It supports links, images, tables, and multimedia content.
- ⑦ HTML works well with CSS and JavaScript to build full websites.

→ 7 disadvantages GSS_Notes

- ① HTML alone cannot create dynamic or interactive websites.
- ② It cannot perform logical operations or calculations.
- ③ HTML depends on CSS and Javascript for design and functionality.
- ④ Large HTML files can become difficult to manage.
- ⑤ It does not provide strong security ~~for~~ features.
- ⑥ HTML cannot ~~connect~~ connect directly to databases.
- ⑦ Different browsers may display HTML pages slightly differently.

• Image Tags

```
<img src = "image.jpg" alt = "Sample Im-  
age" >
```

```
<img src = "image.jpg" alt = "Sample Image"  
width = "300" height = "200" >
```

• Anchor Tags

```
<a href = "https://www.google.com" >  
visit Google </a>
```

```
<a href = "https://www.wikipedia.org"  
target = "- blank" > open wikipedia  
in New Tab </a>
```

• Lists and Types of Lists in HTML

1) Ordered List ()

2) Unordered List ()

- Used when order does not matter.

Example:

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
<title> HTML Lists </title>
```

```
</head>
```



```
<body>
  <h2> Ordered List </h2>
  <ol>
    <li> HTML </li>
    <li> CSS </li>
    <li> Java Script </li>
  </ol>
  <h2> Unordered List </h2>
  <ul>
    <li> Apple </li>
    <li> Banana </li>
    <li> Mango </li>
  </ul>
</body>
</html>
```

• PRACTICAL Example:

```
<!DOCTYPE html>
<head>
  <title> HTML List Tags </title>
</head>
<body>
  <h1> HTML List Tags </h1>
  <!-- ordered list Examples -->
  <h2> Ordered List </h2>
  <h3> 1. Default Ordered List </h3>
  <ol>
    <li> HTML </li>
    <li> CSS </li>
    <li> JavaScript </li>
```


<h3> 2. Ordered List (Uppercase Letters)

</h3>

<ol type = "A">

 C

 C++

 Java

<h3> 3. Ordered List (lowercase Roman)

</h3>

<ol type = "i">

 First

 Second

 Third

GSS Notes

<h3> 4. Ordered List (start Attribute)

</h3>

<ol start = "5">

 Item Five

 Item Six

 Item Seven

<hr>

<!-- UNORDERED List Examples -->

<h2> Unordered List </h2>

<h3> 1. Default Unordered List </h3>

 Apple

```
<li> Banana </li>
<li> Mango </li>
</ul>
```

```
<h3> 2. Unordered List (Circle) </h3>
<ul type = "circle">
  <li> Red </li>
  <li> Green </li>
  <li> Blue </li>
</ul>
```

```
<h3> 3. Unordered List (square) </h3>
<ul type = "square">
  <li> Pen </li>
  <li> Pencil </li>
  <li> Eraser </li>
</ul>
<hr>
```

```
<! -- DESCRIPTION LIST -->
<h2> Description List </h2>
<dl>
  <dt> HTML </dt>
  <dd> Hyper Text Markup language
    </dd>
  <dt> CSS </dt>
  <dd> Cascading Style sheets </dd>
</dl>
```

```
<h2> Nested List </h2>
<ul>
```



```
<li> Frontend </li>  
  <ul type = "circle">  
    <li> HTML </li>  
    <li> CSS </li>  
  </ul>  
</ul>
```

```
<ul>  
  <li> Backend </li>  
  <ol>  
    <li> PHP </li>  
    <li> Node.js </li>  
  </ol>  
</ul>
```

Table Tags in HTML (normal, colspan and rowspan)

Common HTML Table Tags

- `<table>` - defines table
- `<tr>` - table row
- `<th>` - table heading
- `<td>` - table data
- `colspan` - merge columns
- `rowspan` - merge rows

Student List

Roll no	Name	Class	City
101	Saroj Shrestha	10 A	Kathmandu
102	Anita Gurung	10 B	Pokhara
103	Ramesh Thapa	10 A	Biratnagar

GSS_Notes

```
<!DOCTYPE html>
```

```
<thead>
```

```
<title> Student list </title>
```

```
<body>
```

```
<h2> Student List </h2>
```

```
<table border = "1" >
```

```
<tr>
```

```
<th> Roll No </th>
```

```
<th> Name </th>
```

```
<th> class </th>
<th> city </th>
</tr>
```

```
<tr>
  <td> 101 </td>
  <td> Saroj shrestha </td>
  <td> 10 A </td>
  <td> Kathmandu </td>
</tr>
```

```
<tr>
  <td> 102 </td>
  <td> Anita Gurung </td>
  <td> 10 B </td>
  <td> Pokhara </td>
</tr>
```

```
<tr>
  <td> 103 </td>
  <td> Ramesh Thapa </td>
  <td> 10 A </td>
  <td> Biratnagar </td>
</tr>
```

```
</table>
</body>
</html>
```


Weekly Timetable with rowspan and colspan

Class Timetable.

	Monday			Tuesday
Time	9-10	10-11	11-12	9-12
9:00-10:00	English	Mathematics	Science	Practical Lab
10:00-11:00	Nepali		Social	
11:00-12:00	Break / Games		Computer	

<HTML>

<head>

<title> Time Table </title>

<body>

<h2> class Timetable </h2>

<table border="1">

<tr>

<th rowspan="2"> Time </th>

<th colspan="3"> Monday </th>

<th> Tuesday </th>

</tr>

<tr>

<th> 9-10 </th>

<th> 10-11 </th>

<th> 11-12 </th>

<th> 9-12 </th>

</tr>

<tr>

<td> 9:00 - 10:00 </td>

<td> English </td>

<td rowspan="2"> Mathematics
</td>

<td> Science </td>

<td rowspan="3"> Practical Lab
</td>

</tr>

<tr>

<td> 10:00 - 11:00 </td>

<td> Nepali </td>

<td> Social </td>

</tr>

<tr>

<td> 11:00 - 12:00 </td>

<td colspan="2"> Break / Games </td>

<td> computer </td>

</tr>

</table>

</body>

</html>

① Table Caption

→ A table caption is a title or description for a table. It appears above the table by default and explains what the table is about.

② Cell spacing

→ Cell spacing is the space between table cells (the gap between borders of cells.)

③ Cell Padding

→ Cell padding is the space between the cell content and the cell border.

E.g.

```
<table border="1" cellpadding="10"
cellpadding="15">
```

```
<caption> Student Marks Details </caption>
```

• Other Attributes of <table>

GSS_Notes

1) Width

- Sets the width of the table.

```
<table width="60%">
```

2) height

- Sets the height of the table.

```
<table height="200">
```

3) align

- Aligns the table on the page.

```
<table align="center">
```


Values : left, center, right

4) ~~bordercolor~~ bordercolor

- Sets the Color of the table border.

`<table border="2" bordercolor="red">`

5) bgcolor

- sets the background color of the table

`<table bgcolor="lightblue">`

• HTML Forms

→ HTML forms are used to collect user input and send it to a server for processing. Forms are commonly used for login pages, registration forms, feedback forms, search boxes, and online surveys.

`<form>` Tag

- The `<form>` tag is the container for all form elements.

Syntax:

`<form action="URL" method="get/post">`

Important Attributes:

> action - Specifies the Server page that processes the form data.

> method - Defines how data is sent (GET or POST).

• Form Controls (Elements)

1) `<input>`

- The `<input>` tag is the most important form element. It creates various input fields.

Common input types:

- > text - Single-line text
- > Password - Hidden text
- > Radio - Select one option
- > checkbox - Select multiple options
- > email, number, date
- > File - File Upload
- > Submit - Submit form
- > reset - Reset form data

2. `<label>`

- Associates text with an input field and improves accessibility.

3. `<text area>`

- Used for multi-line text input, such as comments or address.

4) `<select>` and `<option>`

- Used to create drop-down lists.

5) `<button>`

- Creates clickable buttons like Submit or reset.

6) <fieldset> and <legend>

- Used to group related form elements and provide a title.

HTML form Example

```
<html>
```

```
<head>
```

```
<title> Complete form example </title>
```

```
</head>
```

```
<body>
```

```
<h2> Student Registration Form </h2>
```

```
<form action = "Submit.php" method = "post">
```

```
<!-- Text input -->
```

```
<label for = "name" > Full Name: </label> <br>
```

```
<input type = "text" id = "name" name = "name">
```

```
<br> <br>
```

```
<!-- Email -->
```

```
<label for = "email" > Email: </label> <br>
```

```
<input type = "email" id = "email" name = "email">
```

```
<br> <br>
```

```
<!-- Password -->
```

```
<label for = "Password" > Password: </label> <br>
```

```
<input type = "Password" id = "Password" name =
```

```
"Password" > <br> <br>
```

```
<!-- Number -->
```

```
<label for = "age" > Age: </label> <br>
```



```
<input type="number" id="age" name="age"><br>
```

```
<!-- Radio buttons -->
```

```
<label> Gender: </label><br>
```

```
<input type="radio" name="gender" value="male">  
male
```

```
<input type="radio" name="gender" value="female"  
checked="TRUE"> Female <br><br>
```

```
<!-- Checkboxes -->
```

```
<label> Skills : </label><br>
```

```
<input type="checkbox" name="Skill 1"> HTML
```

```
<input type="checkbox" name="Skill 2"> CSS
```

```
<input type="checkbox" name="Skill 3"> JS <br>
```

```
<!-- Date -->
```

GSS_Notes

```
<label for="dob"> Date of Birth: </label><br>
```

```
<input type="date" id="dob" name="dob"><br>
```

```
<!-- File Upload -->
```

```
<label for="file"> Upload Resume: </label><br>
```

```
<input type="file" id="file" name="resume"><br>
```

```
<!-- Select dropdown -->
```

```
<label for="Course"> Course: </label><br>
```

```
<select id="Course" name="Course">
```

```
<option> computer Science </option>
```

```
<option> IT </option>
```

```
<option> Electronics </option>
```

```
</select><br><br>
```

<!-- Tentarea -->

<label for="address"> Address : </label>

<textarea id="address" rows="4" cols="40">
</textarea>

<!-- Fieldset & legend -->

<fieldset>

<legend> login Details </legend>

<label> Username: </label>

<input type="text">

<label> Password: </label>

<input type="password">

</fieldset>

<!-- Buttons -->

<input type="submit" value="submit">

<input type="reset" value="Reset">

<button type="button"> click Me </button>

</form>

</body>

</html>

HTML5 Tags: Audio, Video, Embed, Canvas, and SVG

→ HTML5 introduced several native elements for embedding multimedia and graphics directly into web pages without relying on plugins like Flash. These include `<audio>`, `<video>`, `<embed>`, `<canvas>`, and `<svg>`.

1. `<audio>` Tag

→ The `<audio>` element embeds sound content, such as music or podcasts, and supports playback controls natively.

`<audio controls>`

`<source src="audio.mp3" type="audio/mpeg">`

Your browser does not support the audio element.

`</audio>`

2. `<video>` Tag

→ Similar to `<audio>`, the `<video>` element embeds video content with native controls.

`<video width="320" height="240" controls>`

`<source src="video.mp4" type="video/mp4">`

Your browser does not support the video tag.

`</video>`

3. <embed> Tag [No closing tag needed]

→ The <embed> tag is used to embed external content (such as PDFs, videos, audio, or other multimedia files) directly inside a web page.

```
<embed src="document.pdf" type="application/pdf" width="500" height="300">
```

4. <canvas> Tag

→ The <canvas> element provides a drawing surface for graphics using Javascript. It's bitmap-based (pixel-level rendering).

5. <svg> Tag

→ The <svg> element embeds Scalable Vector Graphics, which are XML-based for shapes, paths, and images that scale without losing quality.

GSS_Notes

* <address> tag

→ The <address> tag is a semantic HTML element that is used to enclose details like email, phone, physical address, or social links for a person, organization, or page section.

Basic syntax

→ <address>

Written by John Doe

Email:

john@example.com
Phone: +1234567890
</address>

* <Frame> Tags

→ HTML frames allow you to split the browser window into multiple sections, each loading a different HTML document.

They used:

- <frameset> instead of <body>
- <frame> for each section

```
<frameset cols="30%, 70%">
  <frame src="menu.html">
  <frame src="content.html">
</frameset>
```

or, rows instead of columns.

* <iframe>

→ An inline frame (<iframe>) embeds another HTML document inside the current page.

```
<iframe
  src="page.html"
  width="600"
  height="400"
  title="Embedded content">
</iframe>
```

*

→ The tag was used in older versions of HTML (HTML 3.2 / HTML 4) to change the text appearance such as font size, color, and face.

```
<font size="6" color="blue" face="Arial">  
</font>
```

CSS (Cascading Style sheets)

→ It is used to style and design HTML web pages - controlling layout, colors, fonts, spacing, animations, responsiveness, and much more.

Why is CSS important?

- Separates content from design
- Makes websites attractive and user-friendly
- Saves time by styling multiple pages at once.
- Improves website performance
- Enables responsive designs (mobile-friendly)

• Types of CSS

→ There are three main types of CSS.

① Inline CSS

- Inline CSS is written inside the HTML tag using the style attribute. It affects only

that specific element.

- Syntam

→ `<tag style = "Property: Value;" >`

Example

```
<!DOCTYPE html >
```

```
<html>
```

```
<head>
```

```
  <title> Inline CSS Example </title>
```

```
</head>
```

```
<body style = "background-color: lightyellow;  
  font-family: Arial;" >
```

```
  <h1 style = "color: blue; text-align: center;" >
```

```
    Inline CSS Example
```

```
  </h1>
```

```
  <p style = "color: green; font-size: 16px;" >
```

```
    This paragraph is styled using inline CSS.
```

```
  </p>
```

```
</body>
```

```
</html>
```

GSS_Notes

2. Internal CSS (Embedded CSS)

→ Internal CSS is written inside the `<style>` tag within the `<head>` section of an HTML document.

It applies styles to one single page.

- Syntam

```
<head>
```

```
<style>
```

Selector {

Property: value;

}

</style>

</head>

Example

<!DOCTYPE html>

<html>

<head>

<style>

body {

background-color: lightyellow;

}

h1 {

color: green;

text-align: center;

}

p {

font-size: 16px;

}

</style>

</head>

~~</html>~~

<body>

<h1> Internal CSS </h1>

<p> This paragraph uses internal CSS.

</p>

</body>

</html>

3. External CSS

→ External CSS is written in a separate CSS file and linked to HTML using the `<link>` tag.

It is the most recommended and professional method.

* CSS File (style.css)

```
body {  
    background-color: #f0f0f0;  
}  
  
h1 {  
    color: navy;  
}  
  
p {  
    font-size: 18px;  
    color: black;  
}
```

GSS_Notes

* HTML File

```
<!DOCTYPE html>  
<html>  
  <head>  
    <link rel="stylesheet" href="style.css">  
  </head>  
  <body>  
  
    <h1> External CSS </h1>  
    <p> This page uses external CSS. </p>  
  </body>  
</html>
```


- Simple Internal CSS using id, class (.) and name selectors

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
<title> simple CSS selectors </title>
```

```
<!-- Internal CSS -->
```

```
<style>
```

```
/* ID Selector */
```

```
#heading {
```

```
    color: blue;
```

```
    text-align: center;
```

```
}
```

```
/* class Selector (.) */
```

```
.box {
```

```
    border: 2px solid black;
```

```
    padding: 10 px;
```

```
    width: 250 px;
```

```
    margin: auto;
```

```
}
```

```
/* Name attribute selector */
```

```
input [name="username"] {
```

```
    border: 2px solid red;
```

```
}
```

```
</style>
```

```
</head>
```

```
<body>
```

```
<h1 id="heading">CSS Selectors Example</h1>
```

```
<div class="box">
```

```
<form>
```

```
  Name:
```

```
<input type="text" name="username">
```

```
<br><br>
```

```
<input type="submit" value="Submit">
```

```
</form>
```

```
</div>
```

```
</body>
```

```
</html>
```

Write down ~~the~~ 10 differences between Frontend and Backend Development.

Frontend Development	Backend Development
① It is the visible part of Website.	It is hidden part of a website.
② Users can see and interact with it.	Users cannot see it directly.
③ Focuses on design and layout.	Focuses on data and logic.
④ Runs in the browser.	Runs on the server.
⑤ Uses HTML, CSS, Java-Script.	Uses python, Java, PHP etc.

⑥ makes website look attractive.

It makes website functional.

~~⑥ It makes website look attractive~~

⑦ It Handles buttons, forms, images.

~~It~~ It handles database and server requests.

⑧ It is related to user experience.

It is related to processing and security.

⑨ It is easier to understand visually.

It requires strong programming knowledge.

⑩ example: Website Homepage design.

Example: Login system and data storage.